数值处理与计算

> 本章大部分内容，已经在其他章节内穿插讲述，此处仅做总结，不再展开赘述。

math模块和数学函数

- 使用方法见第2章：基础知识。下表总结math库的常用方法和常量。

功能	描述
ceil(x)	返回大于或等于x的最小整数。
copysign(x, y)	返回带有y符号的x
fabs(x)	返回x的绝对值
factorial(x)	返回x的阶乘
floor(x)	返回小于或等于x的最大整数
fmod(x, y)	当x除以y时返回余数
frexp(x)	返回x的尾数和指数作为对（m，e）
fsum(iterable)	返回迭代器中值的准确浮点和
isfinite(x)	如果x既不是无穷大也不是NaN（不是数字），则返回True
isinf(x)	如果x是正或负无穷大，则返回True
isnan(x)	如果x是NaN，则返回True
ldexp(x, i)	返回x *（2 ** i）
modf(x)	返回x的小数和整数部分
trunc(x)	返回x的截断整数值
exp(x)	返回e ** x
expm1(x)	返回e ** x-1
log(x[, base])	将x的对数返回底数（默认为e）
log1p(x)	返回1 + x的自然对数
log2(x)	返回x的以2为底的对数
log10(x)	返回x的以10为底的对数
pow(x, y)	返回x的幂y
sqrt(x)	返回x的平方根
acos(x)	返回x的反余弦

功能	描述
asin(x)	返回x的反正弦
atan(x)	返回x的反正切
atan2(y, x)	返回atan（y / x）
cos(x)	返回x的余弦
hypot(x, y)	返回欧几里得范数sqrt（x * x + y * y）
sin(x)	返回x的正弦
tan(x)	返回x的切线
degrees(x)	将角度x从弧度转换为度
radians(x)	将角度x从度转换为弧度
acosh(x)	返回x的反双曲余弦值
asinh(x)	返回x的反双曲正弦值
atanh(x)	返回x的反双曲正切值
cosh(x)	返回x的双曲余弦值
sinh(x)	返回x的双曲余弦值
tanh(x)	返回x的双曲正切
erf(x)	返回x处的误差函数
erfc(x)	返回x处的互补误差函数
gamma(x)	返回x处的Gamma函数
lgamma(x)	返回x处Gamma函数绝对值的自然对数
pi	数学常数，圆的周长与其直径之比（3.14159 ...）
e	数学常数e（2.71828 ...）

random模块和随机函数

- 使用方法见第3章和第5章中的相关章节介绍

常用的随机函数总结介绍

1. random.random(): 返回随机生成的一个浮点数，范围在[0,1)之间
2. random.uniform(a, b): 返回随机生成的一个浮点数，范围在[a, b)之间
3. random.randint(a,b): 在指定范围[a,b]内获取一个随机整数
4. random.randrange(a, b, step=c): 在指定范围[a,b)内，在指定的基数和步长值形成的集合中获取1个随机数值

5. random.choice(x): 从指定的序列x中随机获取一个数据
6. random.shuffle(x): 将一个列表x中的元素打乱，随机排序（俗称：洗牌）
7. random.seed(): 设定随机种子

日期和时间：time和datetime模块

time模块

Python的`time`模块提供了各种与时间相关的函数，包括获取当前时间、处理时间间隔、执行时间测量等。以下是对`time`模块中一些常用功能的详细介绍、用法、常见用法以及示例代码和运行结果。

获取当前时间

定义：

- UTC：协调世界时，即世界标准时间，采用24小时制。协调世界时以英国格林尼治的本初子午线上的平太阳时为基准。
- Epoch：Unix纪元，即1970年1月1日00:00:00 UTC。
- UNIX时间戳：Unix time, POSIX time 或 Unix timestamp，是从Epoch（1970年1月1日00:00:00 UTC）开始所经过的秒数，不考虑闰秒。

常用函数：

- time.time(): 返回自Epoch纪元以来的秒数，通常称为Unix时间戳。
- time.localtime(): 返回一个表示本地时间的time.struct_time对象。
- time.gmtime(): 返回一个表示协调世界时（UTC）的time.struct_time对象。

```
import time

# 获取当前时间戳
timestamp = time.time()
print("当前时间戳:", timestamp)

# 获取本地时间
local_time = time.localtime()
print("本地时间:", local_time)

# 获取UTC时间
utc_time = time.gmtime()
print("UTC时间:", utc_time)
```

运行结果（仅示意，具体结果依运行时间而不同）

```
当前时间戳: 1715833304.1631322
本地时间: time.struct_time(tm_year=2024, tm_mon=5, tm_mday=16, tm_hour=12, tm_min=21,
tm_sec=44, tm_wday=3, tm_yday=137, tm_isdst=0)
UTC时间: time.struct_time(tm_year=2024, tm_mon=5, tm_mday=16, tm_hour=4, tm_min=21,
tm_sec=44, tm_wday=3, tm_yday=137, tm_isdst=0)
```

时间格式化

主要解决`time.struct_time`对象与字符串相互转化的问题。

- time.strftime(format, time_tuple): 将time.struct_time对象格式化为字符串。
- time.strptime(string, format): 将字符串解析为time.struct_time对象。

```
import time

# 格式化本地时间
formatted_time = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime())
print("格式化后的本地时间:", formatted_time)

# 解析时间字符串
parsed_time = time.strptime("2021-09-10 10:22:47", "%Y-%m-%d %H:%M:%S")
print("解析后的时间:", parsed_time)
```

运行结果（仅示意，具体结果依运行时间而不同）

```
格式化后的本地时间: 2024-05-16 12:22:02
解析后的时间: time.struct_time(tm_year=2021, tm_mon=9, tm_mday=10, tm_hour=10, tm_min=22,
tm_sec=47, tm_wday=4, tm_yday=253, tm_isdst=-1)
```

datetime模块

示例1：获取当前日期和时间

```
import datetime

datetime_object = datetime.datetime.now()
print(datetime_object)
```

运行结果（仅示意，具体结果依运行时间而不同）

```
2025-09-27 15:03:59.223290
```

我们使用now()方法创建一个包含当前本地日期和时间的datetime对象。

示例2：获取当前日期

```
import datetime

date_object = datetime.date.today()
print(date_object)
```

当您运行程序时，输出将类似于：

```
2025-09-27
```

在这个程序中，我们使用了date类中定义的today()方法来获取一个包含当前本地日期的date对象。

datetime模块中常用的类

- date类
- time类
- datetime类
- timedelta类

示例3：表示日期的date对象

```
import datetime

d = datetime.date(2019, 4, 13)
print(d)
```

运行该程序时，输出为：

```
2019-04-13
```

示例4：获取当前日期

您可以使用一个名为today()的类方法来创建一个包含当前日期的date对象。方法如下:

```
from datetime import date

today = date.today()

print("当前日期:", today)
print("当前年:", today.year)
print("当前月:", today.month)
print("当前日:", today.day)
```

运行结果可能是：

```
当前日期: 2025-09-27
当前年: 2025
当前月: 9
当前日: 27
```

示例5：表示时间的时间对象time

```
from datetime import time

# time(hour = 0, minute = 0, second = 0)
a = time()
print("a =", a)

# time(hour, minute and second)
b = time(11, 34, 56)
```

```
print("b =", b)

# time(hour, minute and second)
c = time(hour = 11, minute = 34, second = 56)
print("c =", c)

# time(hour, minute, second, microsecond)
d = time(11, 34, 56, 234566)
print("d =", d)
```

运行该程序时，输出为：

```
a = 00:00:00
b = 11:34:56
c = 11:34:56
d = 11:34:56.234566
```

示例6：datetime对象

datetime模块有一个名为的datetime类，可以包含来自**date**和**time**对象的信息。

```
from datetime import datetime

#datetime(year, month, day)
a = datetime(2019, 11, 28)
print(a)

# datetime(year, month, day, hour, minute, second, microsecond)
b = datetime(2019, 11, 28, 23, 55, 59, 342380)
print(b)
```

运行该程序时，输出为：

```
2019-11-28 00:00:00
2019-11-28 23:55:59.342380
```

datetime()构造函数中的前三个参数year、month和day是必需的。

示例7：打印年，月，时，分和时间戳

```
from datetime import datetime

a = datetime(2019, 12, 28, 23, 55, 59, 342380)
print("年 =", a.year)
print("月 =", a.month)
print("日 =", a.day)
print("时 =", a.hour)
print("份 =", a.minute)
print("时间戳 =", a.timestamp())
```

运行该程序时，输出为：

```
年 = 2019
月 = 12
日 = 28
时 = 23
份 = 55
时间戳 = 1577548559.34238
```

示例8：两个日期和时间之间的时差timedelta

timedelta对象表示两个日期或时间之间的时差。

```
from datetime import datetime, date

t1 = date(year = 2018, month = 7, day = 12)
t2 = date(year = 2017, month = 12, day = 23)
t3 = t1 - t2
print("t3 =", t3)

t4 = datetime(year = 2018, month = 7, day = 12, hour = 7, minute = 9, second = 33)
t5 = datetime(year = 2019, month = 6, day = 10, hour = 5, minute = 55, second = 13)
t6 = t4 - t5
print("t6 =", t6)

print("type of t3 =", type(t3))
print("type of t6 =", type(t6))
```

运行该程序时，输出为：

```
t3 = 201 days, 0:00:00
t6 = -333 days, 1:14:20
type of t3 = <class 'datetime.timedelta'>
type of t6 = <class 'datetime.timedelta'>
```

注意，t3和t6都是<class 'datetime.timedelta'>类型。

示例9：两个timedelta对象之间的时间差

```
from datetime import timedelta

t1 = timedelta(weeks = 2, days = 5, hours = 1, seconds = 33)
t2 = timedelta(days = 4, hours = 11, minutes = 4, seconds = 54)
t3 = t1 - t2

print("t3 =", t3)
```

运行该程序时，输出为：

```
t3 = 14 days, 13:55:39
```

在这里，我们创建了两个timedelta对象t1和t2，它们直接相差的天数被打印在屏幕上。